

Algorithmen und Datenstrukturen

Übung 1 – Einführung

Aufgabe 1: Begrifflichkeiten

a) Gegeben sei das folgende, ausschnittsweise dargestellte C-Programm.

```
int main(/* ... */)
{
    char username[512];
    gets(username);           // gets ist eine SCHLECHTE Idee!

    char password[512];
    gets(password);           // gets ist eine SCHLECHTE Idee!

    char previous_login_error_count_str[512];
    load_login_error_count_from_table
        (username, previous_login_error_count_str);
    int previous_login_error_count =
        atoi(previous_login_error_count_str);

    if (previous_login_error_count >= 3)
    {
        printf("Wegen zu vielen Versuchen gesperrt.");
        return -1;
    }

    /* ... */

    return 0;
}
```

Beantworten Sie die folgenden Teilfragen:

- Welche Daten gibt es in dem Programmfragment?
- Welche Repräsentation besitzen die Daten?
- Hat ein Datum mehrere Repräsentationen?
- Wie wirkt es sich auf Daten und Repräsentationen aus, wenn man das Programm mehrmals parallel anstartet?
- Warum ist gets eine **schlechte** Idee?

b) Angenommen, im Rahmen einer Big Data-Aufgabe sollen unternehmensweit über viele Standorte Daten ausgetauscht werden.

- Warum ist es wichtig, sich in diesem Fall auf die Repräsentation der Daten zu einigen?
- An welcher Stelle muss eine konventionsgemäße Datenrepräsentation mindestens eingehalten werden?
- Welche Rolle spielt in diesem Fall eine Datenaufbereitung?

c) Zum Baum auf Folie 16:

- Wie sieht der Baum aus, wenn man statt dem Wort krautsalat das Wort bratwurst oben ("in der Wurzel") stehen haben möchte?
- Ist über das Wort in der Wurzel eindeutig festgelegt, wie der Baum aussieht? Falls ja: Warum? Falls nein: Geben Sie ein Gegenbeispiel an.
- Welche Bedingung müssen die Daten erfüllen, dass alle Äste im Baum (d. h. die Wege von der Wurzel bis ganz unten) gleich lang sind?

Aufgabe 2: Formalisierung

a) Angenommen, Sie wollen den Effekt eines analogen Lautstärkereglers (z. B. mit Drehwiderstand wie an einem Lautsprecher) mathematisch formulieren. Sie müssen dazu eine Funktion angeben, die zu jeder **beliebigen** Eingabelautstärke die durch den Regler verringerte Lautstärke ausgibt.

Geben Sie im Rückblick auf die Mathematikvorlesungen des Studiums eine entsprechende Funktion soweit möglich an. Die Abbildungsvorschrift darf in diesem Rahmen entfallen, weil Sie die Abbildung nicht kennen. Wählen Sie aber passende Definitions- und Zielmengen für die Funktion.

Hinweis: Hier geht es primär um die Notation, nicht um die Physik!

b) Die mathematische Funktion aus Teilaufgabe a) ist auf dem Rechner keinesfalls perfekt (z. B. als Signatur einer C-Funktion) umzusetzen.

- Worin besteht das Problem?
- Wie müssen Sie die mathematische Funktionsdefinition anpassen, damit die Umsetzung am Rechner (ohne Tricks) möglich wird?

c) Gegeben seien zwei Einzeldaten $x \in \mathbb{R}$ und $y \in \mathbb{N}^+$. Mathematisch lässt sich aus diesen Einzeldaten auf zwei konzeptuell unterschiedliche Weisen ein zusammengesetztes Datum konstruieren: $z_1 = (x, y)$ und $z_2 = \{x, y\}$.

- Worin unterscheiden sich z_1 und z_2 ?
- Welche Variante ist im Allgemeinen geschickter?
- Was ist die Basismenge für z_1 ?
- für Profis: Was ist die Basismenge für z_2 ?

d) Geben Sie formal korrekt die Menge aller möglichen terminierten ASCII-Strings an. Dazu gehen Sie wie folgt vor:

- Geben Sie eine Menge M_1 an, die als Datentyp für ein einzelnes ASCII-Zeichen geeignet ist.
- Das kartesische Produkt $M_A \times M_B$ ergibt aus zwei Mengen M_A und M_B eine neue Menge mit allen möglichen Reihungen zweier Tupeln beider Mengen.

Zum Beispiel ist $\mathbb{R}^2 = \mathbb{R} \times \mathbb{R}$ die Menge aller Punkte mit zwei Koordinaten. Nutzen Sie diese Überlegung, um die Menge aller Strings mit exakt zwei Zeichen mathematisch zu notieren.

- Erweitern Sie ihre Überlegungen auf die Menge aller Strings mit exakt drei, mit exakt vier, mit exakt fünf, ... Zeichen.
- Angenommen, Sie haben die Mengen aller Strings mit exakt einem, mit exakt zwei, mit exakt drei, mit exakt vier Zeichen. Was müssen Sie machen, um eine Menge mit allen Strings zu bekommen, die zwischen eins und vier Zeichen beinhalten?
- Für alle möglichen Strings fehlt nur noch ein "..." an der richtigen Stelle.

e) Übersetzen Sie den folgenden C-Code für eine struct (d. h. für einen Datentyp) in eine formal korrekte, mathematische Notation – d. h. in eine Menge, so dass jedes Element der Menge eine mögliche Instanz der struct abbildet.

```

1  typedef struct obstacle_
2  {
3      int x_position, y_position;
4      char* name;                // string name attached
5
6  } obstacle;
```

Die vorangehende Aufgabe kann nützlich sein. Verwenden Sie ansonsten M_S als Menge aller möglichen ASCII-Strings.

Aufgabe 3: Einfache Datenrepräsentation

a) Erklären Sie am Beispiel der hexadezimalen Bytefolge CA FF EE 00, was der Unterschied zwischen einer big-endian und einer little-endian-Repräsentation ist. Konvertieren Sie dazu die Bytefolge beides Mal in eine dezimale, menschenlesbare Variante, unter der Annahme, dass ein vorzeichenloser Integer gespeichert ist.

b) Geben Sie die Zahl -107 als hexadezimale Bytefolge im Sinne folgender Datenrepräsentationen an:

- ein signed int gemäß big-endian- und two's-complement-Konventionen.
- ein signed int gemäß little-endian- und one's-complement-Konventionen.
- ein Array mit ASCII-Zeichen.

c) Angenommen, ein C-Programm verwendet den nachfolgenden Code, um ein Datum in eine Datei zu speichern und um das Datum an anderer Programmstelle (z. B. in einem späteren Durchlauf auf einem anderen Rechner) erneut einzulesen.

```

// speichern:
int value = /* ... vom Programm sinnvoll befüllt ... */;
FILE *f = fopen("network_share/test.bin", "wb");
fwrite(f, &value, sizeof(int));
```

```

/* ... andere Stelle im Programm, später, anderer Rechner ... */

// Laden:
int value;
FILE *f = fopen("network_share/test.bin", "rb");
int value;
fread(f, &value, sizeof(int));

printf("%i", value);

```

Der Netzwerkshare sei in beiden Fällen verfügbar, die Datei lasse sich ohne Fehler (z. B. Verbindungsabbruch) byteweise einlesen.

Beantworten Sie die folgenden Fragen:

- Warum tritt eventuell ein Problem auf? Wie äußert sich das Problem? Was für eine Ausgabe ergibt sich?
- Wie wäre das Problem zu beheben, wenn man auf die hardwarenahe Datenrepräsentation beschränkt ist?
- Wie könnte man das Problem beheben, wenn man in der Datei eine beliebige Repräsentation des Datums wählen dürfte? Wo liegt der Vorteil, der Nachteil dieser Variante?

d) Der Wechsel zwischen einer hardwarenahen und einer maschinenfernen Datenrepräsentation ist bisweilen recht teuer. Führen Sie selbst einen entsprechenden Test in einem C-Programm durch:

- Befüllen Sie vorab ein `float`-Array (z. B. `float array[...]`) mit extrem vielen (z. B. mehreren Millionen) zufälligen `float`-Werten zwischen 0 und 1.
- Wandeln Sie in einer `for`-Schleife alle Einträge des Arrays in Zeichenfolgen um und speichern Sie die Zeichenfolgen anschließend in einem passend vor dimensionierten Array (z. B. `char* array[...]`) ab.
- Gehen Sie in einer weiteren Schleife alle Einträge des Zeichenfolgenarrays durch, wandeln Sie diese Einträge in `floats` um, und überschreiben Sie die Einträge im `float`-Array.
- Ziehen Sie zum Vergleich eine dritte Schleife heran, welche abwechselnd `floats` addiert und subtrahiert – die also `arr[1] + arr[2] - arr[3] + arr[4] - arr[5]` und so weiter ausrechnet.
- Messen Sie, wie viel Zeit jede Schleife in Anspruch nimmt. Welche Schleife ist am teuersten? Wie viel teurer ist eine Konvertierung als eine einfache Rechenoperation?
- für Profis: Wo geht die Zeit bei den Konvertierungen verloren? Ein Profiler wäre das geeignete Werkzeug, um entsprechende Daten zu sammeln.

Hinweise:

- Der Compiler optimiert reine Performancetests mal mehr, mal weniger gut aus dem Programm heraus. Ein Zurückschreiben temporärer Variablen in ein Array ist ein guter Weg, um das zu verhindern.

- Seien Sie vorsichtig mit Testausgaben: Ein Herausschreiben mittels `printf` kann zum Beispiel auf die nächste Bildschirmaktualisierung (z. B. 10 ms) warten, was bei vielen Ausgaben einen Großteil der Rechenzeit einnimmt.

e) Schreiben Sie eine C++-Funktion, welche beliebige `float`-Werte (d. h. beliebige Bit-Belegungen in einer `float`-Variable) in eine Zeichenfolge überführt – und zwar derart, dass die Abbildung umkehrbar ist: Aus der Zeichenfolge soll sich exakt (d. h. Bit für Bit identisch) der ursprüngliche `float`-Wert rekonstruieren lassen, egal wie groß oder klein dieser war.

Prüfen Sie ihre Funktion, indem Sie auch eine passende Rückkonvertierung implementieren. Dann muss `to_float(to_string(v)) == v` für jedes beliebige `v` erfüllt sein. Treten Sie einen Beweis an, indem Sie tatsächlich jedes mögliche `v` durch die Funktion schicken und die obige Bedingung prüfen!

Hinweis: Nur weil es eine Zeichenfolge ist, braucht deren Inhalt nicht für Menschen verständlich sein. Zur Inspiration können Sie z. B. nachschlagen, was eine BASE64-Kodierung ist, auch wenn es diese nur für Integer gibt.

f) für Profis: Wie in der vorangehenden Teilaufgabe, nur: Der `string`, welchen die Konvertierung zurückliefert, soll jetzt eine für Menschen annehmbare Darstellung besitzen, z. B. als Kommazahl oder in Exponentialschreibweise. Spezielle `float`-Werte (z. B. NAN, INF) dürfen als fester Text herauskommen.